



OOPS — Interactive Revision



How to use this page

1. Read the plain-English idea first.
2. Look at the code.
3. Before opening any **Predict** toggle, say your answer out loud or write it down. Then open it and check.
4. Tick the section off in the tracker only when you got the predictions right without peeking.

Based on your own Day 6–14 notebooks in the OOPS folder. Examples use the **Book** theme so everything connects. The older recap page is here: ♦ [OOPS](#)

✓ Progress tracker

1. [Class vs Object](#)
2. [Constructor \(`\_\_init\_\_` \)](#)
3. [`self`](#)
4. [Attributes and Methods](#)
5. [Encapsulation: public / protected / private](#)
6. [Getters and Setters](#)
7. [`@property`](#)
8. [Inheritance and `super\(\)`](#)
9. [Multilevel and Multiple inheritance](#)
10. [Polymorphism](#)
11. [Abstraction \(ABC\)](#)
12. [Magic methods: `\_\_str\_\_` and `\_\_repr\_\_`](#)
13. [Operator overloading](#)
14. [Composition \(has-a\)](#)
15. [Bonus: Custom exceptions are classes too](#)



[Your real mistakes log](#)

 [One-glance cheat sheet](#)

 [Practice questions \(we'll solve these together next\)](#)

[Easy](#)

[Medium](#)

[Hard](#)

Progress tracker

- ☒ ~~Part 1: Class vs Object~~
 - ☒ ~~Part 2: Constructor (`__init__`)~~
 - ☒ ~~Part 3: `self`~~
 - ☒ ~~Part 4: Attributes and Methods~~
 - ☒ ~~Part 5: Encapsulation (public / protected / private)~~
 - ☐ Part 6: Getters and Setters
 - ☐ Part 7: `@property`
 - ☐ Part 8: Inheritance and `super()`
 - ☐ Part 9: Multilevel and Multiple inheritance (MRO)
 - ☐ Part 10: Polymorphism
 - ☐ Part 11: Abstraction (ABC)
 - ☐ Part 12: Magic methods `__str__` and `__repr__`
 - ☐ Part 13: Operator overloading
 - ☐ Part 14: Composition (has-a)
 - ☐ Part 15: Bonus, custom exceptions are classes too
-

1. Class vs Object

Plain idea: A **class** is a blueprint. An **object** is one real thing built from that blueprint.

Analogy: "Book" as an idea says every book has a title and an author. *Harry Potter* sitting on your desk is one actual book. The idea is the class. The real book is the object.

```
class Book:
    pass

book1 = Book()    # one real book
book2 = Book()    # another real book

print(type(book1))
print(book1 == book2)
```

▼ 🧠 **Predict:** what do the two prints show?

`<class '__main__.Book'>` and then `False`. Both came from the same blueprint, but they are two separate objects living at two different places in memory.



`Book` (no brackets) = the blueprint. `Book()` = using the blueprint to build one. `book1` = the thing you built.

2. Constructor (`__init__`)

Plain idea: `__init__` runs **automatically** the moment an object is built. Its job: "this object is not allowed to exist without these details."

Analogy: A library won't put a book on the shelf until it has a title and author written on it.

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

b1 = Book("Atomic Habits", "James Clear")
b2 = Book()
```

▼ 🧠 **Predict:** what happens on the last line?

`TypeError: Book.__init__() missing 2 required positional arguments: 'title' and 'author'`. The constructor demands both, and you gave none.

▼  **Predict:** how would you make `author` optional?

Give it a default: `def __init__(self, title, author="Unknown"):`. Now `Book("X")` works and `author` becomes `"Unknown"`. You did exactly this in your `BankAccount` with `balance = 0`.

3. `self`

Plain idea: `self` means "the object that is being worked on right now." Many objects share the same class code. `self` tells Python whose data to use.

Analogy: A form letter says "Dear ". The template is the same for everyone. When it's printed for you, the blank becomes your name. `self` is that blank.

```
b1 = Book("Atomic Habits", "James Clear")
b2 = Book("Dune", "Frank Herbert")
print(b2.title)
```

▼  **Predict:** when Python builds `b2`, what is `self` inside `__init__`? What prints?

`self` is `b2`. So `self.title = title` means `b2.title = "Dune"`. It prints `Dune`. It can never mix up `b1` and `b2`, because `self` is always the object on the left of the dot.



Trap: every method needs `self` as its first parameter. Forget it and calling `b1.describe()` gives `TypeError: describe() takes 0 positional arguments but 1 was given`, because Python secretly passes `b1` in.

4. Attributes and Methods

Plain idea: **Attributes** = data an object holds (`title`, `author`). **Methods** = things an object can do (functions that live inside the class).

```
class Book:
    def __init__(self, title, author):
        self.title = title
```

```

        self.author = author

    def rate(self, rating):
        self.rating = rating          # saved ON the object
        print(f"{self.title} rated {rating}/5")

b1 = Book("Dune", "Frank Herbert")
b1.rate(5)
print(b1.rating)

b2 = Book("Emma", "Jane Austen")
print(b2.rating)

```

▼ 🧠 **Predict:** what do the last two prints do?

`print(b1.rating)` → `5`, because `rate()` stored it with `self.rating`.

`print(b2.rating)` → `AttributeError`. `b2` never called `rate()`, so it never got a `rating`. Each object has its own separate data.

▼ 🧠 **Predict:** you run `b1.pages = 400`, then `print(b1.pages)` and `print(b2.pages)`. What happens?

`b1.pages = 400` works fine, Python lets you attach new attributes anytime. But `b2.pages` is still an `AttributeError`. Adding to one object never adds to the others. (You saw this in Day 6 with `audi.windows = 4` and then `tata.windows` failing.)



Two traps you actually hit:

- Printing a parameter is not the same as storing it. `print(rating)` shows it once; `self.rating = rating` keeps it.
- Don't give a method and an attribute the same name. If the method is `rating` and you also do `self.rating = 5`, the number overwrites the method on that object.

5. Encapsulation: public / protected / private

Plain idea: Control who can touch an object's data, so nobody can put nonsense in it.

Analogy: Your bank balance. You can check it and deposit through the app, but you can't just type a new number into the bank's database.

Style	Example	Meaning
Public	<code>self.title</code>	Anyone can read and change it
Protected	<code>self._title</code>	A polite "please don't touch from outside". Child classes are expected to use it. Python does NOT block it.
Private	<code>self.__title</code>	Python renames it to <code>_Book__title</code> so outside code can't reach it by the normal name

```
class Book:
    def __init__(self, title, price):
        self.title = title
        self.__price = price

b = Book("Dune", 500)
print(b.title)
print(b.__price)
```

▼ 🧠 **Predict:** which print fails and why?

`print(b.title)` → `Dune` . `print(b.__price)` → `AttributeError: 'Book' object has no attribute '__price'` . Python secretly renamed it to `_Book__price` (called **name mangling**). That's why your crash earlier said `_Book__rate` .

6. Getters and Setters

Plain idea: If data is private, you give people **controlled doors**: a getter to read it, a setter to change it with rules.

```

class Book:
    def __init__(self, title, price):
        self.title = title
        self.__price = price

    def get_price(self):
        return self.__price

    def set_price(self, price):
        if price <= 0:
            print("Price must be positive")
        else:
            self.__price = price

b = Book("Dune", 500)
b.set_price(-20)
b.set_price(650)
print(b.get_price())

```

▼ 🧠 **Predict:** full output?

Price must be positive then 650. The -20 was rejected by the setter's rule, so the price never became negative.



Trap from your Day 8 notebook: `print(person.sett_age())` crashed with `missing 1 required positional argument: 'age'`. A setter needs a value to set. To read, call the getter.

7. @property

Plain idea: Same protection as getters/setters, but it *looks* like a normal attribute. You write `b.price`, not `b.get_price()`.

```

class Book:
    def __init__(self, title, price):
        self.title = title

```

```

        self.price = price                # goes THROUGH the setter

    @property
    def price(self):                      # runs on: b.price
        return self.__price

    @price.setter
    def price(self, value):               # runs on: b.price = value
        if value <= 0:
            raise ValueError("Price must be positive")
        self.__price = value

b = Book("Dune", 500)
b.price = 650
print(b.price)

```

▼  **Predict:** does `b.price = 650` run the getter or the setter? What prints?

The **setter** (it's an assignment). It passes the check, so `650` prints.

▼  **Predict:** what if `__init__` said `self.__price = price` instead of `self.price = price`, and you did `Book("Dune", -9)`?

The object is created with price `-9`, no error. Writing to `self.__price` directly **skips the setter**, so no validation ran. This is the exact bug you found in your EBook code.

▼  **Predict:** in your version the setter did `return 'not a valid rate'` instead of raising. What happens with a negative value?

The setter returns early, so `__rate` is never created. The object exists but is half-built. Reading `e2.rate` later crashes with `AttributeError: 'Ebook' object has no attribute '_Book__rate'`. Lesson: in a setter, **raise an error** for bad values instead of returning a string. The return value of a setter is thrown away anyway.

8. Inheritance and `super()`

Plain idea: A child class gets everything the parent has for free, and adds its own extras. It's an **"is-a"** relationship: an EBook *is a* Book.

Why: no copy-pasting. Fix a bug in Book once, every child gets the fix.

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def describe(self):
        print(f"{self.title} by {self.author}")

class EBook(Book):
    def __init__(self, title, author, size_mb):
        super().__init__(title, author) # let Book set up
title + author
        self.size_mb = size_mb          # then add my own
extra

e = EBook("Dune", "Frank Herbert", 3.2)
e.describe()
print(e.size_mb)
```

▼  **Predict:** EBook never wrote `describe()`. Does `e.describe()` work?

Yes: `Dune by Frank Herbert` then `3.2`. EBook inherited `describe()` from Book.

▼  **Predict:** what if you delete the `super().__init__(...)` line?

The EBook's own `__init__` fully replaces Book's, so `title` and `author` are never set. `e.describe()` crashes with `AttributeError: 'EBook' object has no attribute 'title'`.



ML link: every PyTorch model starts with `class MyNet(nn.Module):` and `super().__init__()`. Same idea.

9. Multilevel and Multiple inheritance

Multilevel = a chain: Book → ComicBook → DCComic. Each level adds something. (Your Day 8 `DC_Comic` example.)

Multiple = one child with two parents at the same time.

```
class Book:
    def __init__(self, title):
        self.title = title

class Sellable:
    def __init__(self, price):
        self.price = price

class ShopBook(Book, Sellable):
    def __init__(self, title, price):
        Book.__init__(self, title)           # call each paren
t by name
        Sellable.__init__(self, price)      # and pass self b
y hand

s = ShopBook("Dune", 500)
print(s.title, s.price)
```

▼ 🧠 **Predict:** if both parents had a method `info()`, which one does `s.info()` use?

`Book` 's. Python checks parents in the order you listed them: `ShopBook(Book, Sellable)` → ShopBook first, then Book, then Sellable. This order is the **MRO** (Method Resolution Order). See it with `ShopBook.__mro__`.

10. Polymorphism

Plain idea: same method name, different behavior depending on the object.

Analogy: Say "make a sound" to a dog: it barks. Same words to a cat: it meows.

```

class Book:
    def read(self):
        return "Turning paper pages"

class EBook(Book):
    def read(self):
        return "Swiping on a screen"

class AudioBook(Book):
    def read(self):
        return "Listening through headphones"

for item in [Book(), EBook(), AudioBook()]:
    print(item.read())

```

▼  **Predict:** output? And why doesn't the loop need any if/else?

Turning paper pages , Swiping on a screen , Listening through headphones .

Each child **overrides** `read()` . Python looks for `read` on the actual object first. It only falls back to the parent's version if the child doesn't have one. So the loop just trusts each object to know how to read itself.

▼  **Predict:** add `class PDFBook(Book): pass` to the list. What does it print?

Turning paper pages . PDFBook didn't override `read()` , so it uses Book's.

11. Abstraction (ABC)

Plain idea: A parent that says "every child **must** write this method." You can't create the parent itself, and you can't create a child that skipped the method.

```

from abc import ABC, abstractmethod

class LibraryItem(ABC):
    @abstractmethod
    def borrow_days(self):
        pass

```

```

def welcome(self):          # normal methods are all
owed too
    print("Welcome to the library")

class Book(LibraryItem):
    def borrow_days(self):
        return 14

class Magazine(LibraryItem):
    pass

```

▼  **Predict:** which of these work? `LibraryItem()`, `Book()`, `Magazine()`

`LibraryItem()` → `TypeError`, can't create an abstract class.

`Book()` → works, it wrote `borrow_days`.

`Magazine()` → `TypeError`, it skipped the required method.

The error happens **when you create the object**, not later. That's the point: mistakes are caught early.

12. Magic methods: `__str__` and `__repr__`

Plain idea: Methods with double underscores on both sides that Python calls **for you** at special moments. `__init__` is one you already know.

- `__str__` → friendly text for users. Used by `print(obj)`.
- `__repr__` → exact text for developers. Used when you just type the variable in a notebook cell.

```

class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):

```

```

        return f"{self.title} by {self.author}"

    def __repr__(self):
        return f"Book('{self.title}', '{self.author}')"

b = Book("Dune", "Frank Herbert")
print(b)
print(repr(b))

```

▼  **Predict:** both outputs? And what prints if you delete both methods?

Dune by Frank Herbert and Book('Dune', 'Frank Herbert').

Without them: something ugly like <__main__.Book object at 0x103cda7b0>.

If only `__repr__` exists, `print()` falls back to using it.

13. Operator overloading

Plain idea: Teach your objects what `+`, `-`, `==`, `<` mean. `a + b` secretly runs

`a.__add__(b)`.

You write	Python actually runs
<code>a + b</code>	<code>a.__add__(b)</code>
<code>a - b</code>	<code>a.__sub__(b)</code>
<code>a * b</code>	<code>a.__mul__(b)</code>
<code>a / b</code>	<code>a.__truediv__(b)</code>
<code>a == b</code>	<code>a.__eq__(b)</code>
<code>a < b</code>	<code>a.__lt__(b)</code>
<code>len(a)</code>	<code>a.__len__()</code>

```

class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __add__(self, other):
        return self.pages + other.pages

```

```
def __lt__(self, other):
    return self.pages < other.pages

a = Book("Dune", 600)
b = Book("Emma", 400)
print(a + b)
print(a < b)
print(a == b)
```

▼ 🧠 **Predict:** all three outputs?

1000, False, False.

The last one is sneaky: without `__eq__`, `==` checks "are these the exact same object in memory?", not "same data?". Two different objects → `False`, even if their data matched.



ML link: this is why `array1 + array2` works element-by-element in NumPy and PyTorch. Someone wrote `__add__` for them.

14. Composition (has-a)

Plain idea: Instead of *being* something (inheritance), an object *holds* another object inside it.

- Inheritance: an EBook **is a** Book.
- Composition: a Library **has** Books. A Library is not a Book.

```
class Book:
    def __init__(self, title):
        self.title = title

class Library:
    def __init__(self):
        self.books = []           # Library HAS a list
of Books
```

```
def add(self, book):
    self.books.append(book)

def show(self):
    for b in self.books:
        print(b.title)

lib = Library()
lib.add(Book("Dune"))
lib.add(Book("Emma"))
lib.show()
print(lib.books[1].title)
```

▼ 🧠 **Predict:** output?

Dune , Emma , then Emma again. `lib.books[1]` is the second Book object, and `.title` reads its title. You chain dots to reach inside: same as `new_home.kitchen_creds.contains()` in your Day 8 notebook.

▼ 🧠 **Quick judgement:** inheritance or composition?

Car and Engine → **composition** (car has an engine).

Manager and Employee → **inheritance** (manager is an employee).

Student and Library card → **composition**.

Test: if the sentence "X is a Y" sounds right, inherit. If "X has a Y" sounds right, compose.

15. Bonus: Custom exceptions are classes too

Plain idea: Your own error types are just classes that inherit from `Exception`. This is where OOP meets exception handling (your Day 14 `DobException`).

```
class LibraryError(Exception):
    pass
```

```
class BookNotAvailable(LibraryError):
    pass

try:
    raise BookNotAvailable("Dune is already borrowed")
except LibraryError as e:
    print("Caught:", e)
```

▼  **Predict:** does `except LibraryError` catch a `BookNotAvailable`?

Yes: `Caught: Dune is already borrowed`. `BookNotAvailable` **is a** `LibraryError` (inheritance), so the parent's `except` catches it. Same rule as `isinstance()`.



Your real mistakes log



These came from your own notebooks and our sessions. Re-read before any test.

1. `class Dog:` with nothing under it → `IncompleteInputError`. An empty class needs `pass`.
2. Calling a setter with no value (`person.sett_age()`) → missing argument error.
3. Reading `person.name` when it was stored as `self.__name` → `AttributeError`.
4. Printing a value inside a method but never storing it on `self`.
5. Method and attribute with the same name → one overwrites the other.
6. `__init__` writing `self.__rate` directly → setter validation skipped.
7. Setter returning a string instead of raising → half-built object, crash later.



One-glance cheat sheet

Concept	One line	Keyword to remember
Class / Object	Blueprint / real thing	<code>Book()</code>
Constructor	Runs automatically on creation	<code>__init__</code>
self	The object being worked on right now	<code>self.x</code>
Encapsulation	Hide data, allow access through controlled doors	<code>__x</code> , getter/setter, <code>@property</code>
Inheritance	Child gets parent's stuff for free (is-a)	<code>class B(A)</code> , <code>super()</code>
Polymorphism	Same method name, different behavior	overriding
Abstraction	Force children to write a method	<code>ABC</code> , <code>@abstractmethod</code>
Magic methods	Python calls them for you	<code>__str__</code> , <code>__repr__</code> , <code>__add__</code>
Composition	Object holds other objects (has-a)	<code>self.engine = Engine()</code>

Practice questions (we'll solve these together next)

No answers here on purpose. Tick them as you solve.

Easy

- ☐ **Q1.** Make a `Student` class with `name` and `marks`. Add a method `is_pass()` that returns True if marks $\geq$ 40.
- ☐ **Q2.** Add `__str__` to Student so `print(s)` shows `Anukrit: 92 marks`.
- ☐ **Q3.** Make `marks` private with a `@property` + setter that rejects anything outside 0–100 by raising `ValueError`.

Medium

- ☐ **Q4.** `BankAccount` with `deposit` and `withdraw`. Withdrawing more than the balance should raise a custom `InsufficientFunds` exception.
- ☐ **Q5.** `Shape` abstract class with abstract `area()`. Build `Circle`, `Rectangle`, `Square(Rectangle)`. Loop over a list and print every area.
- ☐ **Q6.** `Money` class with `amount`. Overload `+`, `==` and `<` so you can add and compare Money objects.

Hard

- ☐ **Q7.** `Library` (composition) holding `Book` objects. Methods: `add_book`, `borrow(title)`, `return_book(title)`, `available()`. Borrowing a missing or already-borrowed book raises your own exceptions.
- ☐ **Q8.** `Employee` → `Manager(Employee)` → `Director(Manager)`. Each level adds one attribute and overrides `describe()` while still calling the parent's version with `super()`.
- ☐ **Q9.** Explain in 2 lines each, without code: why `@property` over plain getters? When composition over inheritance? What does `super()` actually do?